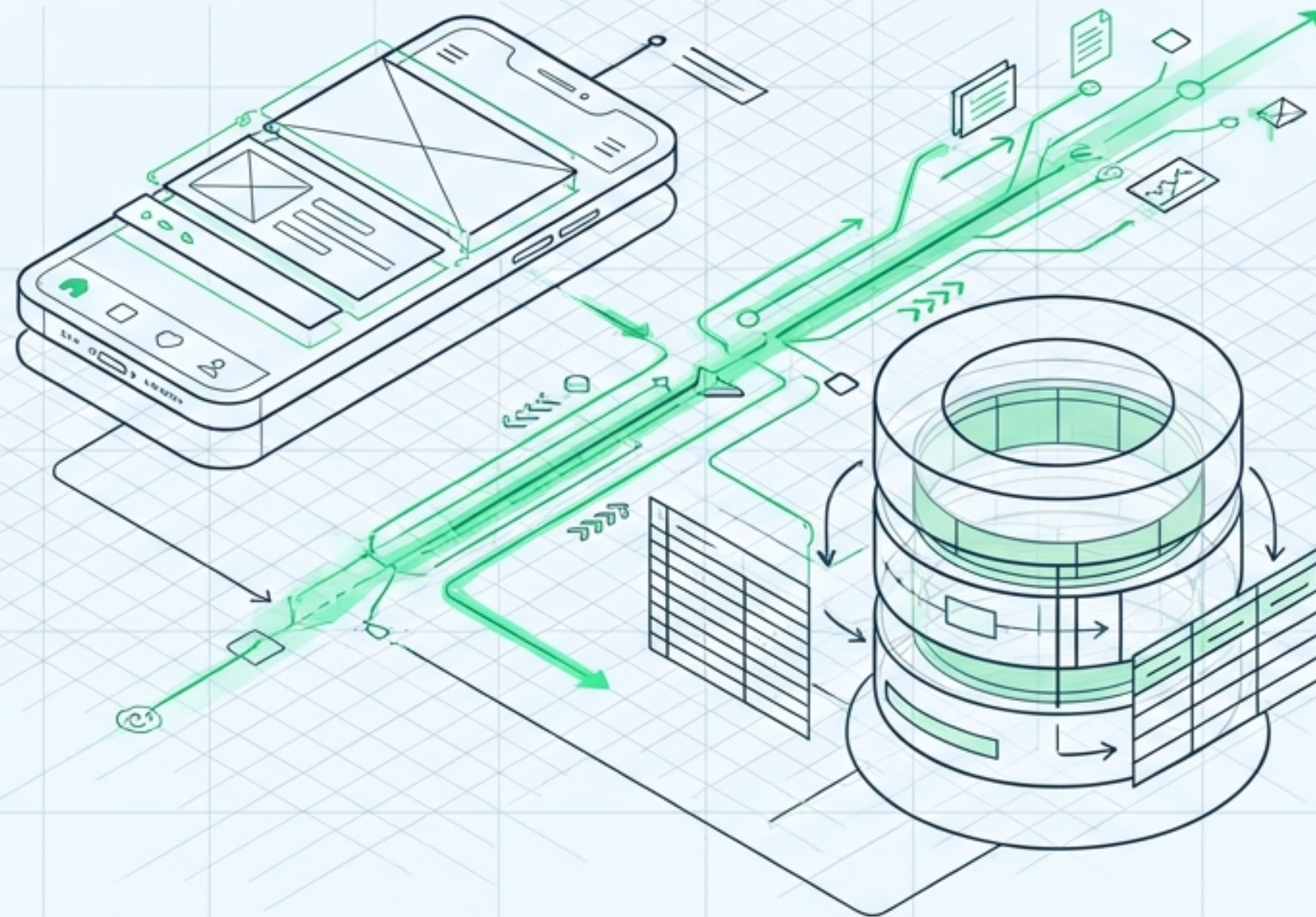


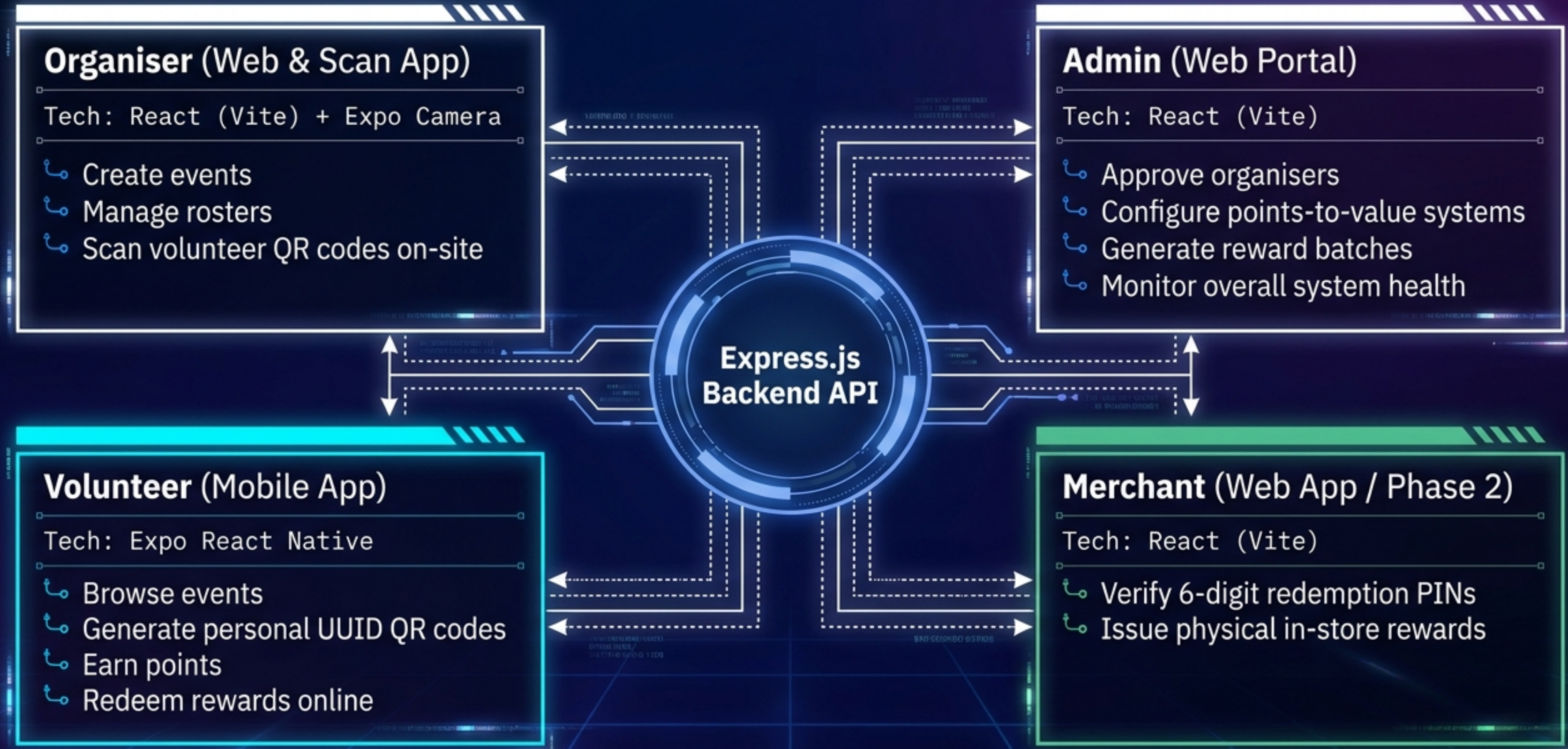
Architecture & Engineering Specification: Volunteering Rewards App

A full-stack software ecosystem built on strict contract-first API design and vertical slice methodology.

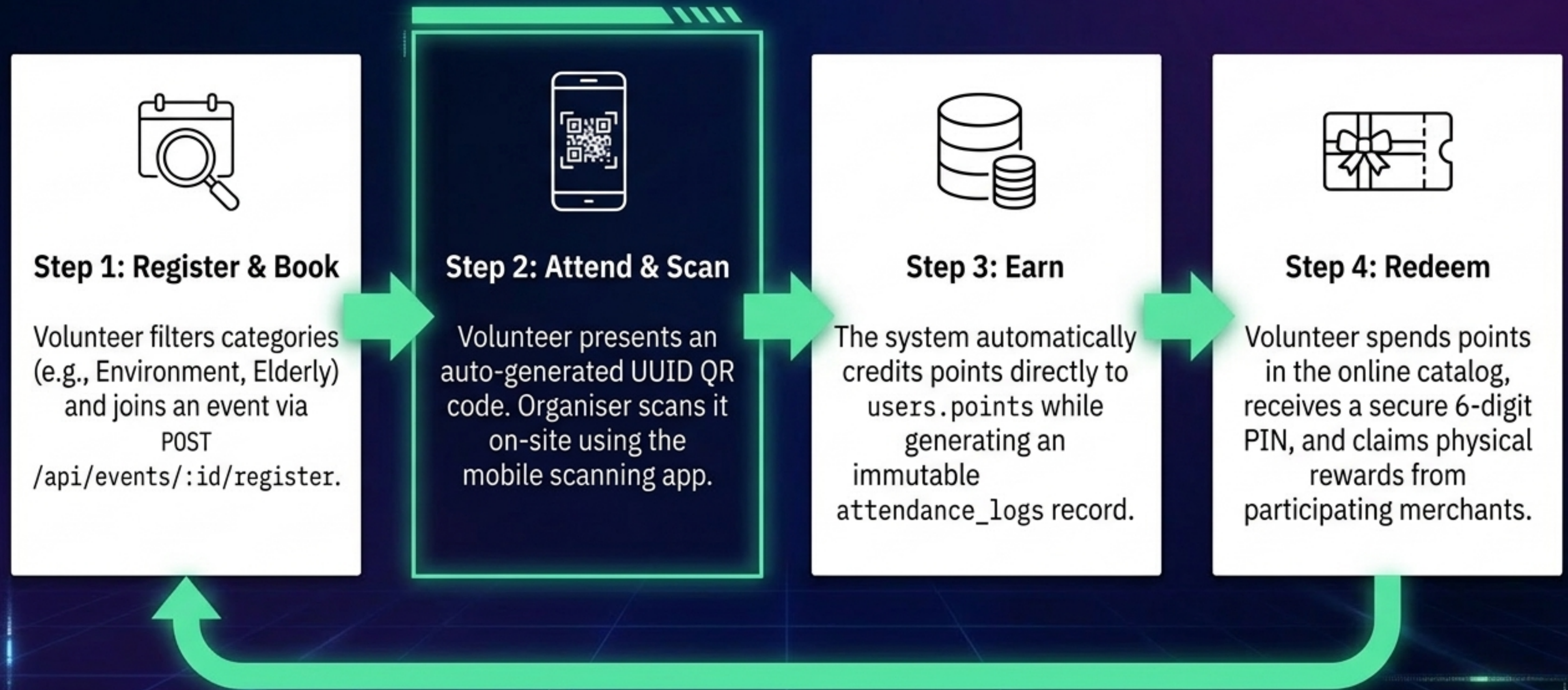


VERSION: 3.0
STATUS: ENGINEERING FREEZE
TARGET RELEASE: JULY 2026

A unified multi-app ecosystem powered by a single Express.js backend



The core volunteering lifecycle creates a self-sustaining engagement loop



Three distinct architectural layers ensure separation of concerns

Client UIs

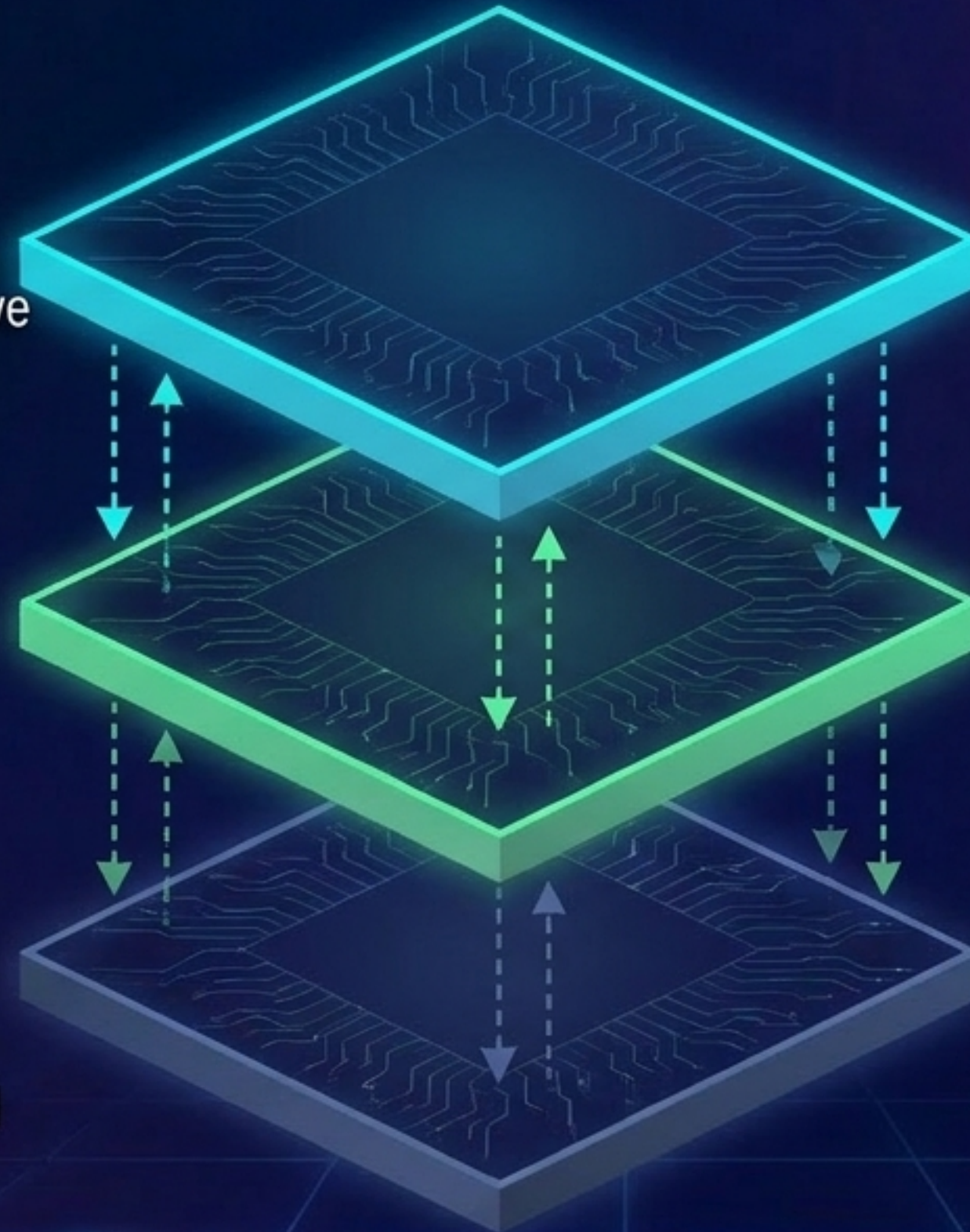
Mobile Apps: Expo / React Native
Web Portals: React / Vite

REST API Server

Node.js / Express.js
Stateless architecture,
custom middleware stack

Database

PostgreSQL
Raw pg queries, parameterized
to prevent injection



KEY_SPECIFICATIONS

- AUTH_SYSTEM: JWT
(Access 15m + Refresh 7d)
- PASSWORD_HASH: bcrypt
(12 salt rounds)
- VALIDATION: Joi / Zod
- DB_MAPPING: No ORM (Raw SQL optimized queries)

59 fully specified API endpoints dictate all client-server communication

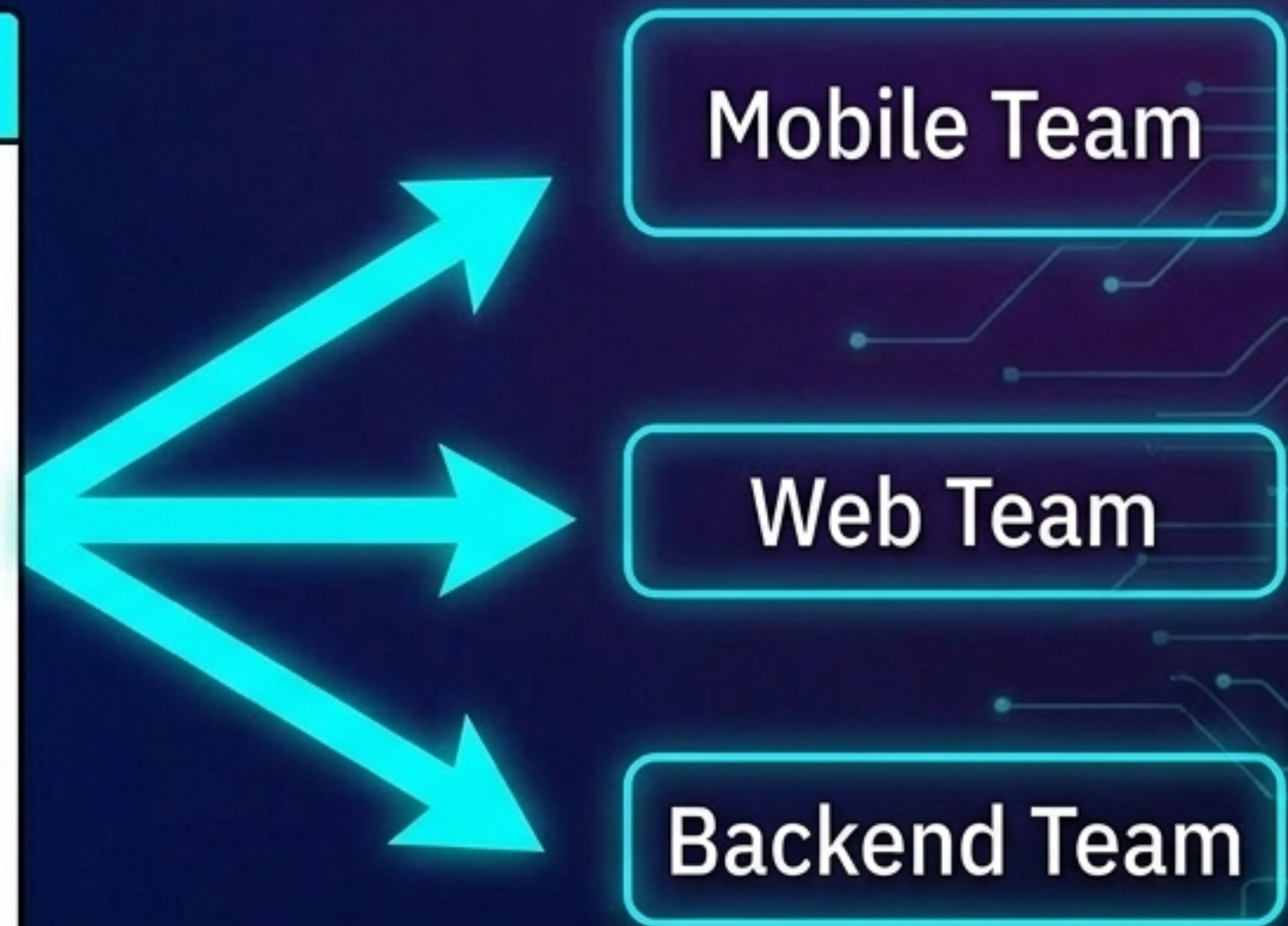
API_CONTRACTS.md

The Golden Rule:

Every request shape, response shape, and HTTP status code was frozen in API_CONTRACTS_v2.md prior to development.

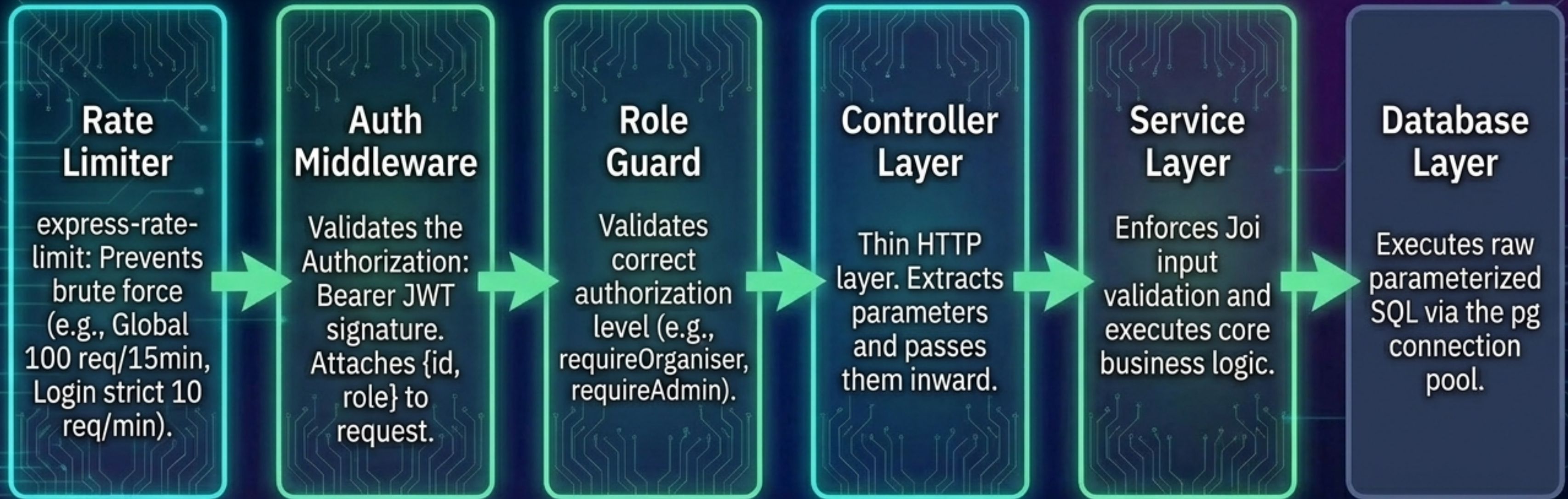
Standardized Error Shapes

```
{
  "error": {
    "code": "validation_error",
    "message": "Human readable description",
    "details": {}
  }
}
```



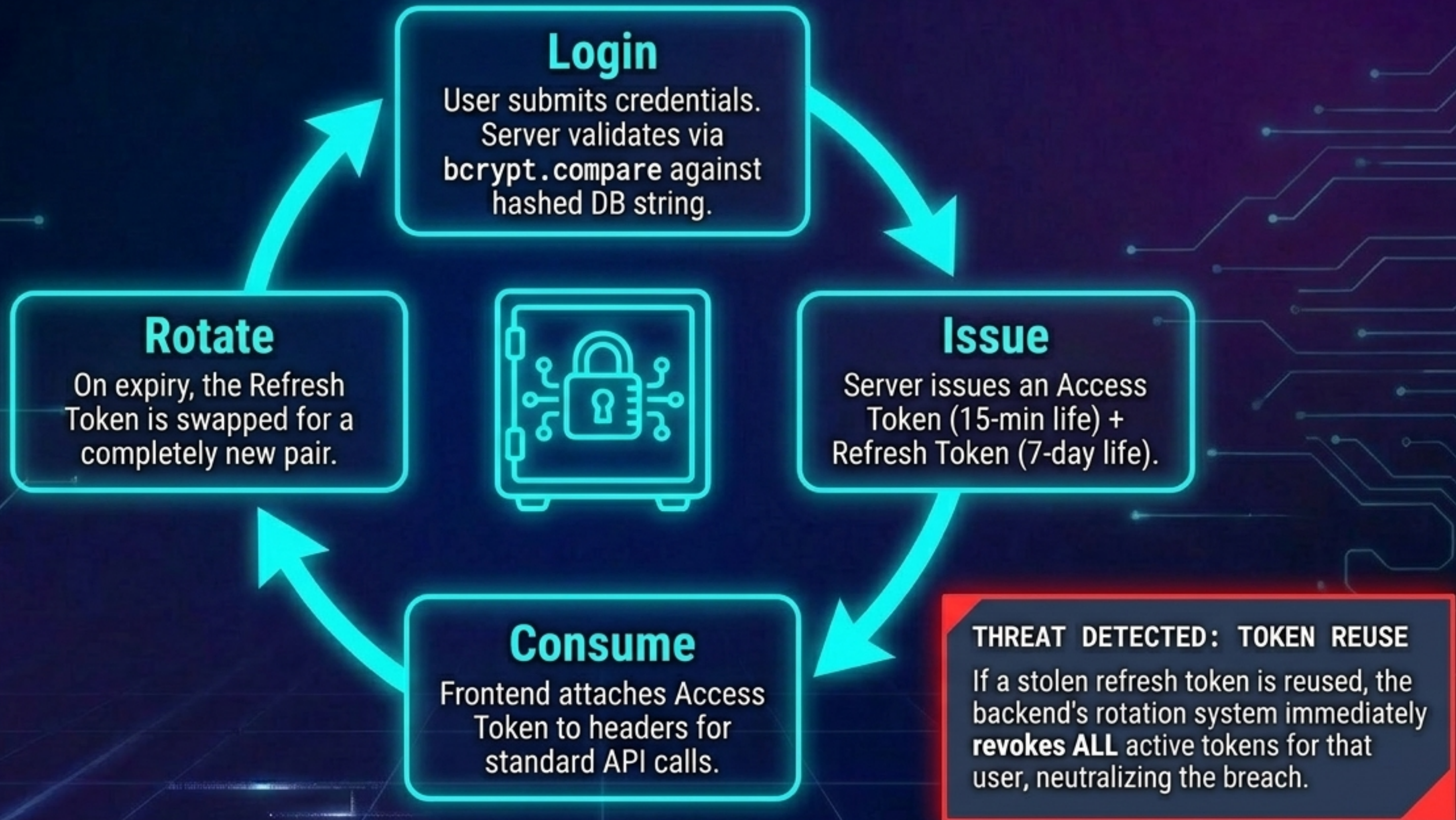
Contract-first design allows frontend and backend developers to work entirely independently with zero integration friction.

Every API request passes through strict middleware security checkpoints



Every protected endpoint passes through up to three security checkpoints before reaching business logic.

A rotating two-token system ensures seamless sessions and high security



12 core PostgreSQL tables organized into three distinct operational clusters



Architectural Decision: Bypassing heavy ORMs for raw parameterized SQL guarantees optimal query performance and exact developer control over data access.

Decoupling the point balance from the audit trail guarantees $O(1)$ read performance

The Traditional Model

- Relies on heavy **points_ledger** append-only tables.
- Requires computationally expensive **SUM()** operations on every profile load.

The Adopted Model

- **users.points** is a simple INTEGER column updated atomically (**UPDATE users SET points = points + X**).
- Balance reads are instant: $O(1)$ performance.
- Auditability remains **perfect**: Earning history lives in **attendance_logs**, spending history lives in **redemption_logs**.

Decoupling the balance value from the audit trail guarantees $O(1)$ read performance for user profiles while maintaining perfect accounting.

Vertical slice methodology guarantees end-to-end full-stack accountability

The Traditional Bottleneck (Horizontal Layers)

Frontend Dev

Backend Dev

Database Dev

Result: Handoff blockers and diluted individual assessment.

The Vertical Slice Solution (Vertical Columns)

1 Developer owns
the complete
feature end-to-end.

Build Frontend
UI screen

Write Controller &
API Route

Write Service logic

Write DB query

Eliminates team bottlenecks, enables independent sprint demonstrations, and guarantees verifiable full-stack contribution evidence for every team member.

Distributed feature ownership across four distinct functional domains

Xon	Vivian	Grace	Nurain
Auth & Core • Backend: Auth routes, JWT middleware, Error handling, DB migrations. • Apps: Admin Web Dashboard, Mobile Auth Flow. • Infrastructure: CI/CD pipelines, Docker deployment.	Events & Logistics • Backend: Event CRUD, Attendance logging. • Apps: Mobile Event Browse, Organiser Scanning App. • Key Tech: expo-camera QR integration.	Rewards & Redemption • Backend: Coupon CRUD, PIN generation & verification. • Apps: Mobile Rewards Catalog, Merchant Web Portal. • Key Tech: PIN Validation.	Admin & Organiser Config • Backend: Dashboard stats, User management, Rewards config. • Apps: Organiser Web Portal, Admin Event configuration.

Strict integration guardrails prevent codebase drift and spaghetti code



Shared AI Prompts

All developers utilize shared, standardized AI prompts that enforce precise syntactic rules (e.g., using `next(err)` instead of throwing raw errors, standardized response formatting).



Strict Table Ownership

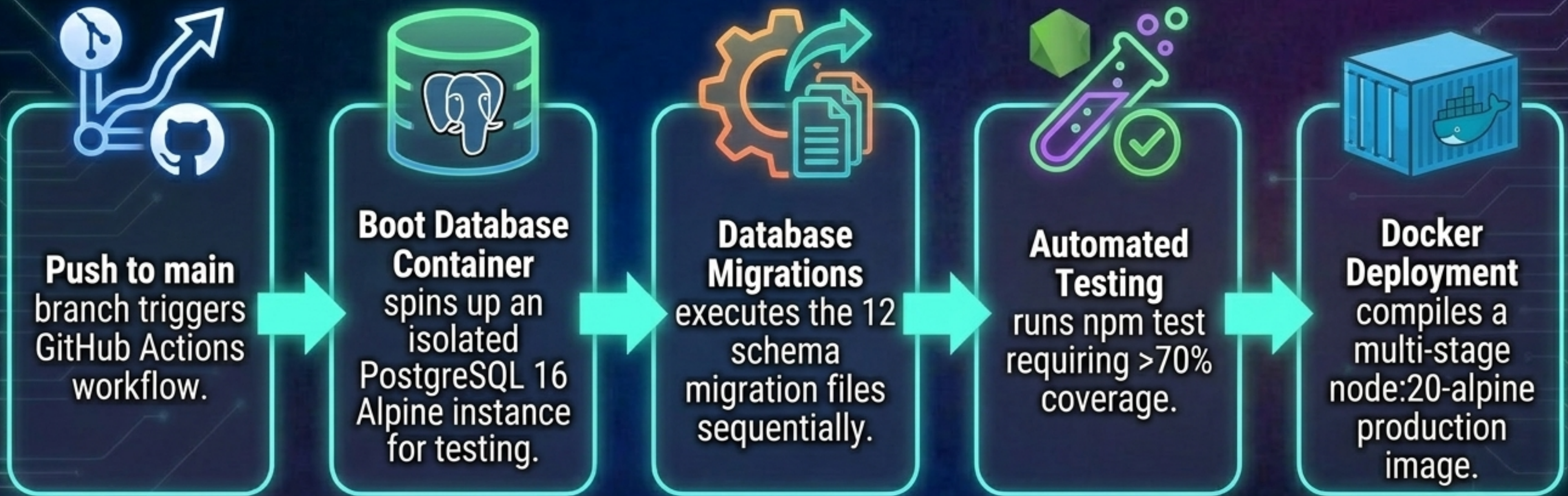
Services can only write to their assigned domain tables. Cross-domain data access is strictly read-only to prevent race conditions.



Mandatory PR Gates

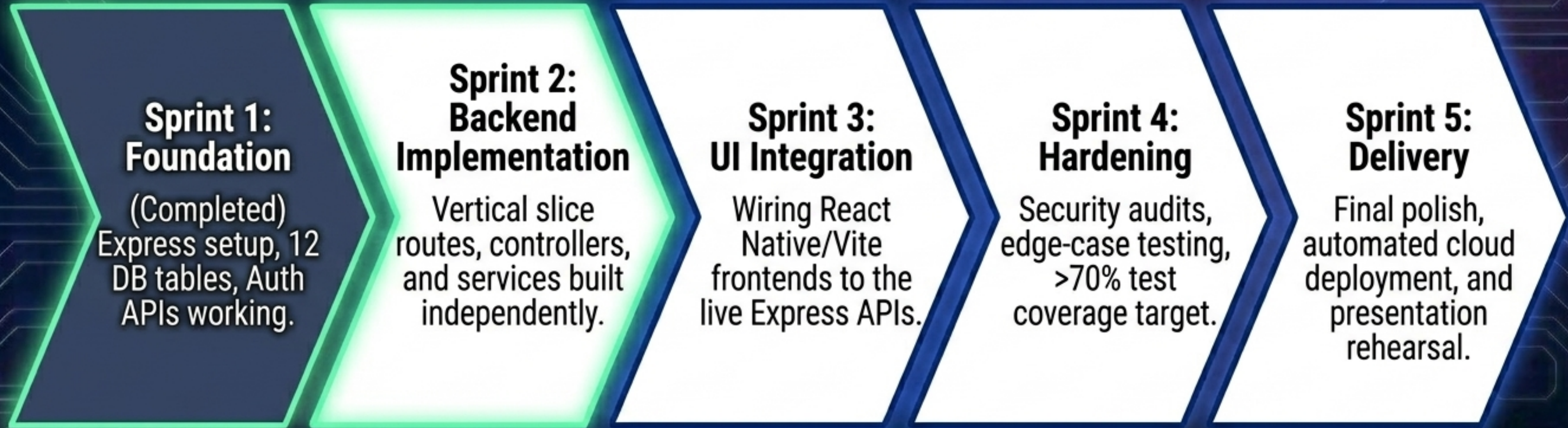
No broken code merges. Requires domain peer review (cross-slice validation), mandatory smoke tests, and two-person approval before Friday merges to main.

Automated CI/CD pipelines and containerization ensure reliable deployment



Any failure during migration or testing instantly breaks the pipeline, blocking the merge and protecting production integrity.

A 60-day, 5-sprint roadmap driving from foundation to cloud deployment



Currently at Sprint 2 checkpoint:
Transitioning to full UI integration.

Project defensibility: Rationale behind critical engineering decisions

Why Express.js?



- Unopinionated and lightweight.
- Middleware composability simplifies complex operations (auth, rate limiting) into reusable stacked functions.

Why Raw SQL over an ORM?



- Guarantees optimal query performance.
- Parameterized queries (\$1) prevent SQL injection natively.

Why Joi Validation?



- Server-side validation is the ultimate defense line against bypassed frontend checks.
- Declarative schemas auto-sanitize data.

Why Vertical Slices?



- Eliminates horizontal development bottlenecks.
- Ensures verifiable individual full-stack contribution for academic grading.